

Lab 5 – Filtering

Name: Mitchell Cuvar

Course/Section: IS-1003-004

Date: 4/13/2026

INTRODUCTION

In this lab, I will be using my VM to apply ethical filters on an open-source data toolset for Kali Linux. I will then search multiple entities using these filters.

BREAKPOINT 1

I installed the Kali Linux toolkit using the Firefox browser the OS came with. After that, I created a new directory in my documents folder called “Lab-05” and double checked in the terminal it was there.

Here is the file located in my Lab-05 folder shown in both the GUI and the terminal:

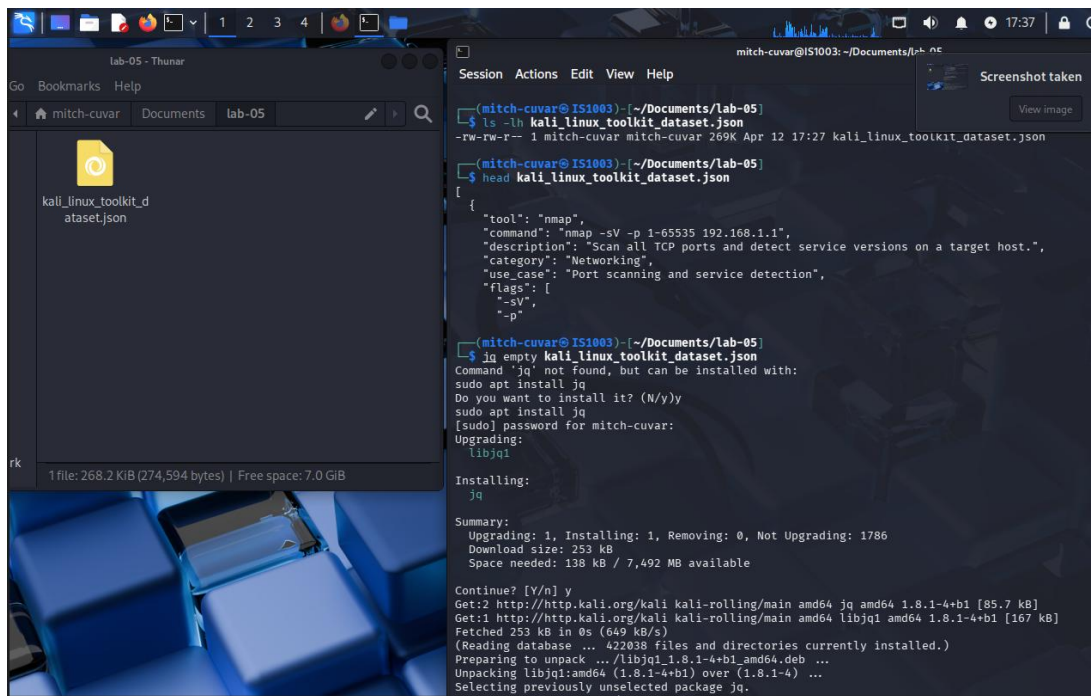


Figure 1: JSON file in Linux GUI and terminal

BREAKPOINT 2

1. Here I have chosen to use this command in Linux: `jq -r '[] | select(.description | test("hash|password"; "i")) | .tool' kali_linux_toolkit_dataset.json`

2. Shown below is the output of that command.

```
(mitch-cuvar@IS1003)~/Documents/lab-05
$ jq -r '.[ ] | select(.description | test("hash|password"; "i")) | .tool' kali_linux_toolkit_dataset.json
hydra
john
hashcat
hydra
john
hashcat
john
hashcat
dcfldd
dcfldd
weeveily
mimikatz
crackmapexec
metasploit
bloodhound
crackmapexec
bloodhound
```

Figure 2: Linux Output 1

3. Here is what I prompted ChatGPT: Explain this command: `jq -r '.[ ] | select(.description | test("hash|password"; "i")) | .tool' kali_linux_toolkit_dataset.json`. Also, explain what `.[ ]` means in a JSON array
4. Here is what GPT responded with:

- `jq`
Processes JSON data
- `-r`
Outputs raw text (no quotes, cleaner output)
- `.[ ]`
Goes through each object in the top-level array one by one
(this is the part people pretend to understand and then absolutely don't)
- `select(...)`
Filters items based on a condition
- `.description | test("hash|password"; "i")`
 - Looks at the `description` field
 - `test("hash|password")` = matches either word
 - `"i"` = case-insensitive
- `| .tool`
After filtering, output the `tool` name
- `kali_linux_toolkit_dataset.json`
The file being processed

What `.[ ]` Means (the important part)

If your JSON looks like:

```
JSON
[
  { "tool": "A", "description": "..."},
  { "tool": "B", "description": "..."}
]
```

Then:

- `.[ ]` = "take the array and give me each object individually"
- Without it, `jq` treats the entire array as one chunk

So:

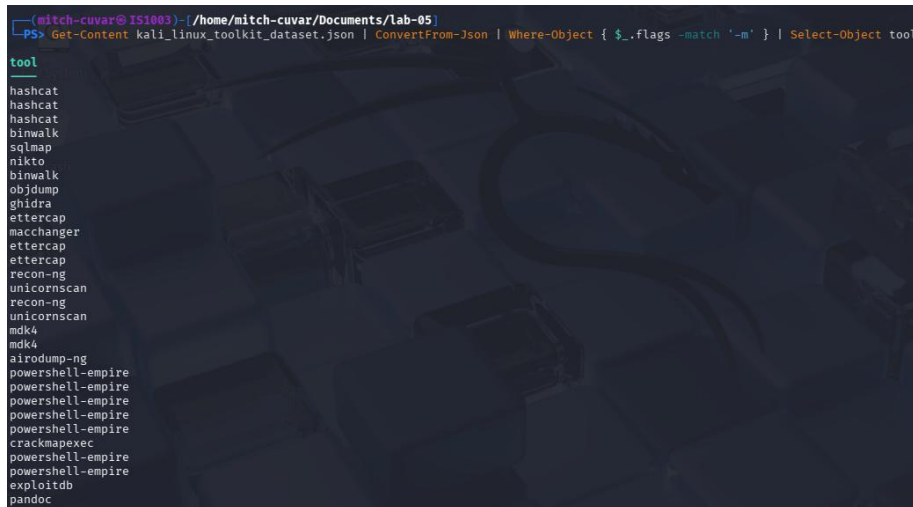
- `.[ ]` → loop through items
- `.tool` → access field in each item

Figure 3: GPT output 1

5. To verify, yes, the AI is correct. In my own words, what this command does is take JSON data and gives you just the raw text from the file (no quotes) and searches through all the entities in the file by each array to find the given keywords in the command. This command lists "hash" and "password," so the `jq` uses `[ ]` to loop through all the objects one array at a time to find everything with those keywords and displays them as raw text.

BREAKPOINT 3

1. Here is the command I chose: `Get-Content kali_linux_toolkit_dataset.json | ConvertFrom-Json | Where-Object { $_.flags -match '-m' } | Select-Object tool`
2. Here is the output of the command:



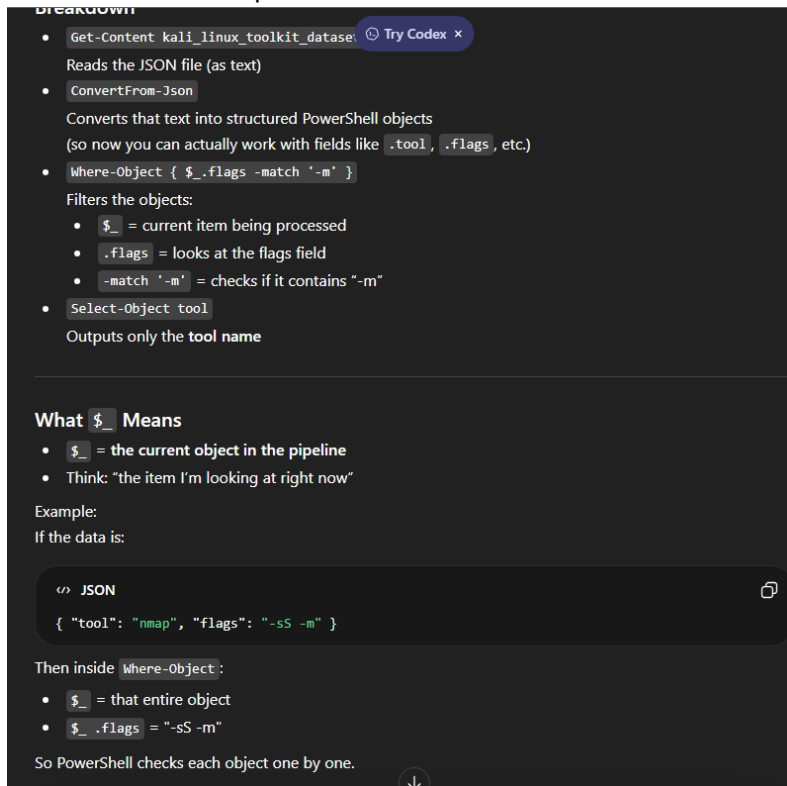
```
(mitch-cuvar@ISI003)-[/home/mitch-cuvar/Documents/Lab-05]
PS> Get-Content kali_linux_toolkit_dataset.json | ConvertFrom-Json | Where-Object { $_.flags -match '-m' } | Select-Object tool

tool
hashcat
hashcat
hashcat
binwalk
sqlmap
nikto
binwalk
objdump
ghidra
ettercap
macchanger
ettercap
ettercap
recon-ng
unicornscan
recon-ng
unicornscan
mdk4
mdk4
airodump-ng
powershell-empire
powershell-empire
powershell-empire
powershell-empire
powershell-empire
crackmapexec
powershell-empire
powershell-empire
exploitdb
pandoc
```

Figure 4: Linux output 2

3. Here is the prompt I used in ChatGPT: Explain this PowerShell command and what `$_` means. `Get-Content kali_linux_toolkit_dataset.json | ConvertFrom-Json | Where-Object { $_.flags -match '-m' } | Select-Object tool`

4. Here is what GPT responded with:



The screenshot shows a GPT chat window with a dark theme. At the top, there's a 'Breakdown' section with a list of PowerShell commands and their functions:

- `Get-Content kali_linux_toolkit_dataset`: Reads the JSON file (as text)
- `ConvertFrom-Json`: Converts that text into structured PowerShell objects (so now you can actually work with fields like `.tool`, `.flags`, etc.)
- `Where-Object { $_.flags -match '-m' }`: Filters the objects:
 - `$_` = current item being processed
 - `.flags` = looks at the flags field
 - `-match '-m'` = checks if it contains "-m"
- `Select-Object tool`: Outputs only the tool name

Below this, a section titled 'What \$_ Means' explains that `$_` is the current object in the pipeline and provides an example JSON object:

```
{ "tool": "nmap", "flags": "-sS -m" }
```

Then, it explains the `Where-Object` command, showing that `$_` is the entire object and `$.flags` is the flags field. It concludes that PowerShell checks each object one by one.

Figure 5: GPT output 2

5. Yes, the AI is correct here. This command is similar to the command I used previously. The biggest difference is that it reads the JSON file data as text already then converts them into PowerShell objects one by one, not by Array. After that, it simply sorts through all the objects (represented by `$_`) in the file and looks at the flags field for each object. It will then check if the flags field matches the keywords in the command and presents them all as individual objects in the PowerShell terminal as just the tool names.

BREAKPOINT 4

1. Here is the command I used: `jq -r '[] | select(.description | test("\b[A-Z]{2,}\b")) | .description'`
`kali_linux_toolkit_dataset.json`

2. Here is the execution of that command:

```
└─$ jq -r '.[ ] | select(.description | test("\\b[A-Z]{2,}\\b")) | .description' kali_linux_toolkit_dataset.json
Scan all TCP ports and detect service versions on a target host.
Perform a stealth SYN scan and OS detection.
Set up a reverse TCP handler for incoming connections.
Brute-force FTP credentials using a password list.
Crack WEP/WPA keys from captured packets.
Capture and analyze TCP traffic on port 80.
Enumerate tables in a database via SQL injection.
Brute-force SSH credentials using user and password lists.
Scan for SMB versions on a network range.
Brute-force HTTP form-based authentication.
Generate a reverse TCP Meterpreter payload for Windows.
Test for SQL injection with POST data and increased detection level.
Scan UDP ports on a target host.
Exploit Windows SMB using EternalBlue.
Brute-force RDP credentials.
Crack NTLM hashes.
Brute-force NTLM hashes with a 4-digit mask.
Scan HTTPS server for vulnerabilities.
Perform aggressive scan with OS and service detection.
Attempt to gain an OS shell via SQL injection.
Brute-force SMTP credentials.
```

Figure 6: Linux output 3

3. Here is the prompt I used in ChatGPT: explain this command `jq -r '.[ ] | select(.description | test("\\b[A-Z]{2,}\\b")) | .description' kali_linux_toolkit_dataset.json`
4. Here is GPT's output:

What it does (simple) Try Codex x

Prints descriptions that contain all-uppercase words (2+ letters) like "API", "DNS", "HTTP".

Breakdown

- `jq`
Processes JSON
- `-r`
Outputs raw text (no quotes)
- `.[ ]`
Loops through each object in the array
- `select(...)`
Filters items based on a condition
- `.description | test(...)`
Looks at the `description` field and checks for a pattern

The Regex (the part everyone pretends to understand)

regex

```
\b[A-Z]{2,}\b
```

- `\b` = word boundary (start/end of a word)
- `[A-Z]` = uppercase letters
- `{2,}` = at least 2 letters

So it matches:

- "API" ✓
- "DNS" ✓
- "A" ✗ (too short)

Figure 7: GPT output 3

5. The AI is correct. This command is pretty simple. All it essentially does is filters out all the objects based on whether it contains capital letters. The prompt asks for at least 2 letters, meaning it will only find objects with at least 2 capital letters next to each other. Then, it lists everything it finds in the terminal. If you want to sort them all alphabetically, simply add "sort" to the end of the command and it will sort everything out alphabetically on its own.

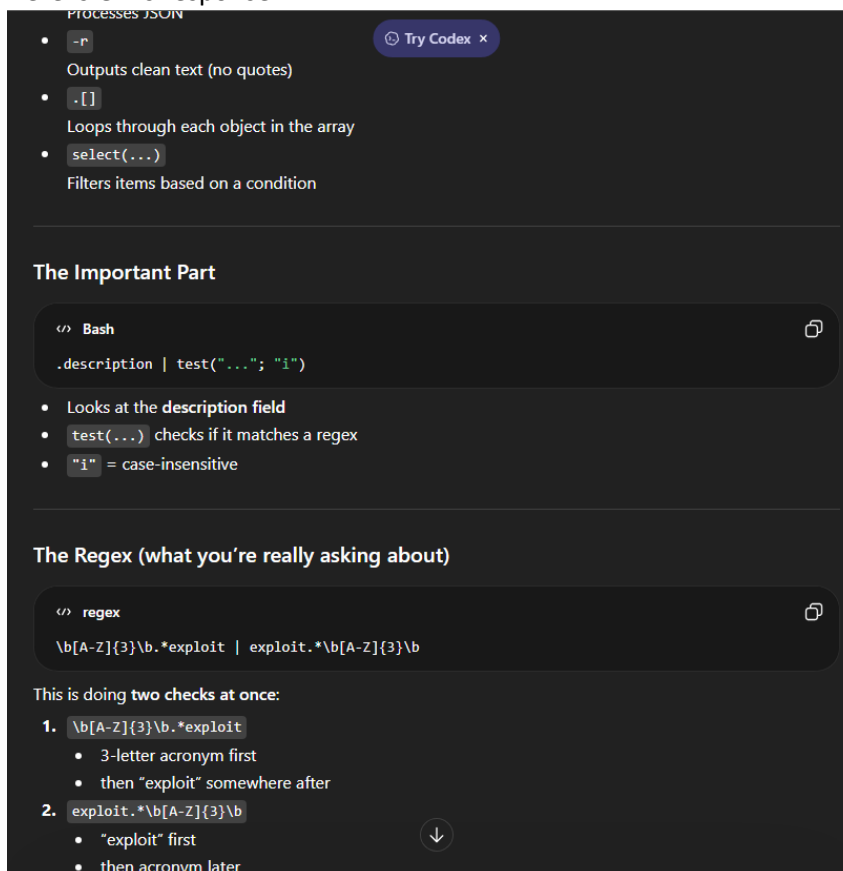
BREAKPOINT 5

1. Here I choose activity 1 and came up with this command: `jq -r '[] | select(.description | test("\\b[A-Z]{3}\\b.*exploit|exploit.*\\b[A-Z]{3}\\b"; "i")) | .tool'`
`kali_linux_toolkit_dataset.json`
2. Here is the execution of this command:

A terminal window with a dark background and a faint cityscape image. The prompt is `(mitch-cuvar@IS1003) ~/Documents/Lab-05`. The command `$ jq -r '[] | select(.description | test("\\b[A-Z]{3}\\b.*exploit|exploit.*\\b[A-Z]{3}\\b"; "i")) | .tool' kali_linux_toolkit_dataset.json` has been executed. The output lists various tools: `metasploit`, `sqlmap`, `metasploit`, `metasploit`, `metasploit`, `powershell-empire`, `metasploit`, `metasploit`, `exploitdb`, `exploitdb`, `metasploit`, `exploitdb`, `metasploit`, `exploitdb`, `metasploit`, `exploitdb`, `burpsuite`, `metasploit`, `exploitdb`, `exploitdb`, and `metasploit`.

Figure 8: Linux output 4

3. Here is the prompt I used in ChatGPT: `jq -r '[] | select(.description | test("\\b[A-Z]{3}\\b.*exploit|exploit.*\\b[A-Z]{3}\\b"; "i")) | .tool'` `kali_linux_toolkit_dataset.json`
4. Here is GPTs response:

A screenshot of a ChatGPT interface. At the top, it says "Processes JSON" and "Try Codex x". Below this, it lists three bullet points:

- `-r`: Outputs clean text (no quotes)
- `[]`: Loops through each object in the array
- `select(...)`: Filters items based on a condition

The Important Part

```
bash
.description | test("..."; "i")
```

- Looks at the **description** field
- `test(...)` checks if it matches a regex
- `"i"` = case-insensitive

The Regex (what you're really asking about)

```
regex
\b[A-Z]{3}\b.*exploit | exploit.*\b[A-Z]{3}\b
```

This is doing **two checks at once**:

1. `\b[A-Z]{3}\b.*exploit`
 - 3-letter acronym first
 - then "exploit" somewhere after
2. `exploit.*\b[A-Z]{3}\b`
 - "exploit" first
 - then acronym later

Figure 9: GPT output 4

5. The AI is correct. This command filters through all the objects in the JSON file and picks out those that have a minimum 3 letter acronym as well as the word “exploit.”

CONCLUSION/LIMITATIONS

- What challenges did you encounter and/or what most interested you about this lab?

What really interested me about this lab was just how in-depth these filters are. Assuming you have basic knowledge on how these commands/filters work it can be a very powerful tool depending on what its used for.

- How did you address your challenges and/or what additional research did you do?

No additional research was needed, the ChatGPT queries broke everything down and explained what I needed to know pretty well.

- How might you use the skills and concepts of this lab going forward?

I honestly have no idea what I personally would use this for, but as I learn more about these things further down the road, I’m sure I could find a (ethical) use.

REFERENCES

[1] R. Mitra, “*Lab-5 Filtering Instructions*,” The University of Texas at San Antonio (2026). Last accessed: 4/13/2026.

GENERATIVE AI SEARCHES

ChatGPT [GPT-5 language model], response to “Explain this command:

`jq -r '[] | select(.description | test("hash|password"; "i")) | .tool' kali_linux_toolkit_dataset.json`

Also explain what `[]` means in a JSON array,” OpenAI. Accessed: April 13th, 2026.

ChatGPT [GPT-5 language model], response to “explain this command `jq -r '[] | select(.description | test("\\b[A-Z]{2,}\\b")) | .description' kali_linux_toolkit_dataset.json`” OpenAI. Accessed: April 13th, 2026.

ChatGPT [GPT-5 language model], response to “explain this command `jq -r '[] | select(.description | test("\\b[A-Z]{2,}\\b")) | .description' kali_linux_toolkit_dataset.json | sort`” OpenAI. Accessed: April 13th, 2026.

ChatGPT [GPT-5 language model], response to “Explain this PowerShell command and what `$_` means. `Get-Content kali_linux_toolkit_dataset.json | ConvertFrom-Json | Where-Object { $_.flags -match '-m' } | Select-Object tool`” OpenAI. Accessed: April 13th, 2026.

COLLABORATION

I worked independently on this lab.